

Customer Churn Intelligence

The whole project, explained in plain English

SDAIM Term 3 · Krishna Mathur (AS25DXB018), Yash Petkar (AS25DXB021), Atharva Soundankar (AS25DXB020) · Instructor: JP Aggarwal

We built a tool that helps a phone company work out which customers are most likely to leave, so staff can look at those accounts first. The tool is finished, it is live online, and anyone can try it right now.

The problem we set out to solve

Phone companies lose customers every month. This is called **churn**. The staff whose job is to keep customers have limited time, so they cannot phone everyone. The question is simple: **which customers should they look at first?**

Right now that choice usually comes down to gut feeling, which is inconsistent and hard to check later. Our tool gives a clear, repeatable answer instead: it looks at a customer's details and says how likely they are to leave, and whether they are worth a closer look.

One thing we were careful about all the way through. The tool only *suggests* who to review. It never decides anything on its own. It does not change prices, cancel contracts, or take any action. A person always makes the final call. We treated that as a rule, not an afterthought.

The data we used

We used a well-known public dataset from IBM. It has **7,043 customers** and **21 pieces of information** about each one, such as how long they have been a customer, what they pay each month, what services they have, and whether they ended up leaving.

Two honest points about this data:

- **It is a made-up company.** The dataset is realistic and widely used for learning, but it does not come from a real business. We say this clearly everywhere, because pretending otherwise would be dishonest.
- **We proved we had the exact right file.** Every file has a unique fingerprint. We check that fingerprint automatically every time the project runs, so nobody can quietly swap in a different file and get different results.

Out of the 7,043 customers, about **27% had left**. That number matters, and it shaped how we measured success later.

What we found when we looked at the data

Before building anything, we explored the data to see what actually drives people to leave. A few patterns stood out clearly:

What we looked at	What we found
Type of contract	People on rolling month-to-month contracts leave far more often (about 43%) than people locked into a two-year deal (under 3%).
How they pay	People paying by electronic cheque leave much more often than people on automatic card payments.
How long they have been a customer	New customers leave much more than long-standing ones. Loyalty builds over time.
Internet type	Fibre customers leave more often than people with slower or no internet.

One caution we kept front of mind: these are **patterns, not causes**. The fact that month-to-month customers leave more does not prove the contract type is the reason. We were careful never to overstate what the numbers show.

The model, and why we trust it

A **model** is just a formula that learns from past customers and then makes a guess about a new one. We trained two different kinds and compared them fairly, plus a very basic one as a sanity check.

The winner was a model called **Logistic Regression**. Here is how well it did, tested on customers it had never seen before:

Question	Answer
Out of everyone who actually left, how many did it catch?	78% (293 of 374)
How good is it overall at telling leavers from stayers?	0.84 out of 1.0
When it flags someone, how often is it right?	About half

We deliberately tuned the model to **catch as many leavers as possible**, even if that means a few false alarms. The reasoning is practical: if we miss a customer who was about to leave, nobody ever reviews them and they are gone for good. A false alarm just costs a staff member a few minutes. Missing someone is the worse mistake, so we built the tool to avoid it.

The part we are most proud of. We decided the rule for picking the winning model *before* we looked at the final test results. This sounds small, but it is what makes our result honest. If you pick the winner after seeing the scores, you are really just choosing the luckiest one. We wrote our rule down in code first, then ran the test once. We even admit in our report that the other model was slightly ahead on a couple of measures, and we still did not switch, because switching after the fact would have broken the whole point.

Checking the model is fair and honest

We went further than the assignment asked, because a tool that affects customers should be checked properly. Three examples:

- **Fairness.** We checked whether the model treats different groups of people evenly. It was even across gender. For senior citizens it behaved differently, but mostly because seniors genuinely leave more often in this data. We measured what would happen if we removed age and gender entirely, and found it barely changed the results, so we recommend removing them.
- **Honest probabilities.** The model tends to sound a bit more certain than it should. We measured exactly how much, and we say so plainly in the tool, so nobody reads too much into the exact percentage.
- **Explaining each answer.** For any single customer, the tool shows which details pushed the score up and which pushed it down. So a staff member does not just get a number, they get a reason to look at.

The dashboard people actually use

We built a clean web page where someone can use the model without needing to understand any of the maths. It does a few things:

- **Check one customer.** Fill in their details, press a button, and get the risk level, a probability, and a suggested action.
- **Check a whole list.** Upload a file of many customers and get them back sorted from most to least at risk, ready to work through. It handles a thousand customers in a fraction of a second.
- **See why.** A chart shows what moved each score.
- **Adjust the sensitivity.** A slider lets the user decide how cautious to be, and everything updates live.
- **Light and dark mode.** A simple toggle, because it is nicer to use.

Getting it online, automatically

A model sitting on a laptop is not much use. We packaged the whole tool so it can run anywhere, and put it online on **Hugging Face**, a popular free hosting service for this kind of app. Anyone can open the link and use it.

The clever part is that we do not update it by hand. We set up an **automatic pipeline**. When we make a change and save it to our code, the following happens on its own:

- The system runs all our checks to make sure nothing is broken.
- If everything passes, it copies the new version up to the live website.
- The website rebuilds itself and starts serving the new version.

We tested this end to end by changing the version number, saving it, and watching the live site update itself with no manual step. It worked. We have the records to prove each run really happened.

We kept it secure. Connecting our code to the website needs a password-like key. We never put that key in our code, where anyone could see it. It is stored safely in a locked settings area, and we even built a scanner that automatically checks nobody accidentally left a key lying around. We tested the scanner by planting fake keys to make sure it actually catches them.

How we made sure it all works

We did not just hope the tool was correct. We wrote **115 automatic tests** that check everything, from whether the data is right to whether the app gives sensible answers. They all pass. We also run a full set of checks before we ever call anything finished. If something breaks, we find out straight away.

Along the way the tests caught several real mistakes that we then fixed, which is exactly what they are for. We would rather show that our safety net actually catches things than pretend we never made an error.

What it does not do, honestly

We think being clear about the limits is part of doing this well.

- The data is from a made-up company, so we cannot promise it would work the same on a real one.
- We have not proven it saves money. To do that properly you would need to try it on real customers with a comparison group, which is beyond this project.
- It spots patterns, it does not prove causes. And a person must always review its suggestions.

So what we can honestly say is this: the tool works, it is live, it is well built and well checked, and it could help a team decide who to look at first. We are careful not to claim more than that.

Try it yourself

Live app: <https://krish21may-churn.hf.space>

Hugging Face Space: <https://huggingface.co/spaces/krish21may/churn>

All the code: <https://github.com/krish2105/SDAIM-Customer-Churn>